

# Kubernetes Pod Pending State SOP

## Objective

Schedule pods successfully when they remain in Pending state because of insufficient resources, affinity rules, taints, PVC binding, or image constraints.

## Scope

Applicable to Kubernetes workloads, platform operations, DevOps teams, SRE teams, application teams, and synthetic incident workflows across development, staging, and production-like environments.

## Pre-requisites

- 1. Access to Kubernetes cluster using kubectl and the correct namespace context.
- 2. Access to application, pod, deployment, service, ingress, and node logs where applicable.
- 3. Access to monitoring dashboards such as Grafana, Prometheus, cloud monitoring, or container insights.
- 4. Permission to inspect manifests, secrets, configmaps, RBAC bindings, PVCs, and rollout history.
- 5. Escalation contact for DevOps, platform, infrastructure, network, and application teams.

## Incident Metadata

| Field               | Value                                           |
|---------------------|-------------------------------------------------|
| Ticket ID           | T031                                            |
| Issue               | Kubernetes pod pending state                    |
| Category            | Deployment                                      |
| Service             | Kubernetes                                      |
| Severity            | High                                            |
| Status              | Resolved                                        |
| Description         | Pod stuck in pending state                      |
| Expected Resolution | Check node resources and scheduling constraints |

## Procedure

- 1. Identify the pending pod, namespace, scheduler events, and requested resources.
- 2. Run kubectl describe pod and review scheduling failure messages.
- 3. Check node capacity, allocatable CPU, memory, GPU, and ephemeral storage.
- 4. Review nodeSelector, affinity, anti-affinity, tolerations, and taints.
- 5. Validate PVC binding status if the pod requires persistent storage.
- 6. Check namespace resource quota and limit range constraints.
- 7. Adjust resource requests, scheduling rules, PVC binding, or cluster capacity.
- 8. Confirm pod is scheduled and transitions to Running.

## Common Commands

```
kubectl config current-context
kubectl get pods -n <namespace> -o wide
kubectl describe pod <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace> --tail=100
kubectl get events -n <namespace> --sort-by=.lastTimestamp
kubectl get deploy,svc,ingress,pvc -n <namespace>
```

## Troubleshooting Matrix

| Issue             | Possible Cause                     | Resolution                      |
|-------------------|------------------------------------|---------------------------------|
| Pod pending       | Insufficient CPU or memory         | Reduce requests or add capacity |
| No matching nodes | Node selector or affinity mismatch | Correct scheduling rules        |
| Untolerated taint | Missing toleration                 | Add approved toleration         |
| PVC pending       | Volume not bound                   | Fix PVC or storage class        |

## Best Practices

1. Set requests based on observed usage.
2. Review taints and tolerations during deployments.
3. Monitor pending pods by namespace.
4. Validate storage binding before workload rollout.

## References

[Kubernetes Docs](#) | [Internal Tickets CSV](#) | [Cluster Events](#) | [Monitoring Dashboards](#) | [Application Logs](#) | [Platform Runbook](#)